Experiment 7

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Dataset Source:

Heart Disease Dataset

Link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

1	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
2	52	1	0	125	212	0	1	168	0	1	2	2	3	0
3	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
4	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
5	61	1	0	148	203	0	1	161	0	0	2	1	3	0
6	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

Dataset Description:

The dataset contains medical attributes used to predict whether a patient has heart disease.

Features:

- **Age**: Age of patient
- **Sex**: Gender (1 = male, 0 = female)
- **Chest Pain Type (cp)**
- **Resting Blood Pressure (trestbps)**
- **Cholesterol (chol)**
- **Fasting Blood Sugar (fbs)**
- **Rest ECG (restecg)**
- **Max Heart Rate (thalach)**
- **Exercise Induced Angina (exang)**
- **Oldpeak**: ST depression
- **Slope**
- **CA**: Number of vessels
- **Thal**

Target Variable:

- **target**
  - 1 → Disease present
  - 0 → No disease

Dataset Characteristics:

- ~300 rows
- 14 columns
- Type: Classification problem

Mathematical Formulation of the Algorithm:

An Artificial Neural Network consists of layers of neurons.

Neuron Operation:

$$z = \sum w_i x_i + b$$

$$a = f(z)$$

Where:

- wi = weights
- xi = inputs
- b = bias
- f = activation function

Activation Functions:

- ReLU:

$$f(x) = \max(0, x)$$

- Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Cross Entropy):

$$L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Algorithm Limitations:

- Requires large data for best performance
- Can overfit easily
- Computationally expensive
- Black-box model (low interpretability)
- Sensitive to hyperparameters

Methodology / Workflow:

Steps:

1. Load dataset

```
df = pd.read_csv('heart.csv')
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

2. Data preprocessing
   - Handle missing values
   - Normalize data

```
X = df.drop('target', axis=1)
y = df['target']

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

3. Split dataset (Train/Test)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

4. Build ANN model using TensorFlow/Keras

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(16, activation='relu', input_dim=X.shape[1]))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

5. Train model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=10,
    validation_split=0.2
)
```

6. Evaluate performance

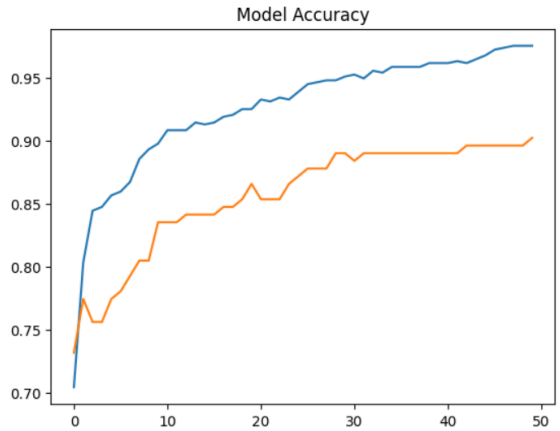
```
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

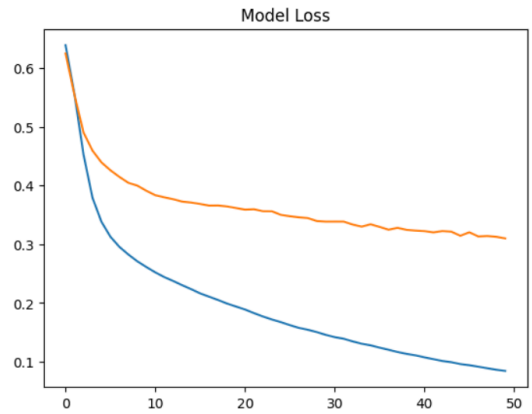
```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9024 - loss: 0.2632
Accuracy: 0.9024389982223511
```

7. Tune hyperparameters

Workflow Diagram:

Dataset → Preprocessing → Train-Test Split →
ANN Model → Training → Evaluation → Prediction





Performance Analysis:

Metrics Used:

- Accuracy
- Confusion Matrix
- Loss Curve

Interpretation:

- High accuracy indicates good prediction capability
- Loss decreases over epochs → model learning properly
- Confusion matrix shows correct/incorrect predictions

Hyperparameter Tuning

Parameters Tuned:

- Number of layers
- Number of neurons
- Learning rate
- Batch size
- Epochs

Impact:

- Increasing neurons → better learning but risk of overfitting
- More epochs → better accuracy (till optimal point)
- Learning rate affects convergence speed

SDG Goal	Implementation in Experiment
SDG 3: Good Health and Well-being	ANN model helps in early prediction of heart disease, supporting better healthcare and reducing mortality risk.
SDG 9: Industry, Innovation and Infrastructure	Use of deep learning models promotes innovation in healthcare technology and intelligent systems.
SDG 8: Decent Work and Economic Growth	Efficient healthcare systems reduce burden on workforce and improve productivity.